

A PROJECT REPORT ON CONTAINER DEPLOYMENT MANAGER

Submitted By

Anshul Sharma – 2315000350

Ankit Tyagi – 2315000314

Ankush Pathak – 2315000324

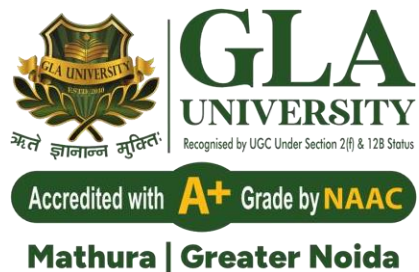
Aonik Yadav – 2315000403

Arjoo – 2315000412

Under the Supervision Of
Mr. Ayush Kumar
GLA University, Mathura

Department of computer science & Engineering

GLA UNIVERSITY, MATHURA



GLA University, Mathura - 281406

DECLARATION

I/we hereby declare that the work which is being presented in the Bachelor of Technology project titled “**Container Deployment Manager System**”, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, and submitted to the Department of Computer Engineering and Applications of **GLA University, Mathura**, is an authentic record of my/our own work carried out under the supervision of **Mr. Ayush Kumar, IntegraMinds Trainer, Dept. of CEA, GLA University**.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign: _____
Name of Candidate: Anshul Sharma
University Roll No.: 2315000350

Sign: _____
Name of Candidate: Ankit Tyagi
University Roll No.: 2315000314

Sign: _____
Name of Candidate: Ankush Pathak
University Roll No.: 2315000324

Sign: _____
Name of Candidate: Aonik Yadav
University Roll No.: 2315000403

Sign: _____
Name of Candidate: Arjoo
University Roll No.: 2315000412

CERTIFICATE

This is to certify that the project entitled “**Container Deployment Manager System**”, carried out in Mini Project – II , is a bonafide work by **Anshul Sharma, Ankit Tyagi, Ankush Pathak, Aonik Yadav, and Arjoo**, and is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (Computer Science & Engineering)**.

Signature of Supervisor: _____

Name of Supervisor: Mr. Ayush Kumar

Date: _____

ACKNOWLEDGMENT

It is a genuine pleasure to express my profound gratitude and deep regard to my guide **Mr. Ayush Kumar** for his valuable guidance, continuous support, and constant encouragement throughout the completion of this project.

I would also like to express my sincere thanks to the faculty and staff of **GLA University, Mathura**, for providing me with the opportunity to work on this project titled “**Container Deployment Manager System.**” This project has helped me gain practical knowledge, strengthen my technical skills, and understand the real-world applications of containerization and full-stack development.

I am also thankful to all my teammates for their cooperation and support, which played an important role in the successful completion of this project.

Thanking You

Sign: _____
Name of Candidate: Anshul Sharma
University Roll No.: 2315000350

Sign: _____
Name of Candidate: Ankit Tyagi
University Roll No.: 2315000314

Sign: _____
Name of Candidate: Ankush Pathak
University Roll No.: 2315000324

Sign: _____
Name of Candidate: Aonik Yadav
University Roll No.: 2315000403

Sign: _____
Name of Candidate: Arjoo
University Roll No.: 2315000412

INDEX	PAGE No.
1. Abstract	01-02
2.Introduction	03-04
3.System Analysis	05-07
4.Objectives and Scope of the Project	08-09
5.Functional Requirements	10-11
6.Use Case Description and Use Case Diagram	12-13
7.System Flow and Interface Design	14-16
8.Hardware & Software Requirements & Future Work	17
9.Conclusion	18
10.Refereneces / Bibliography	19

ABSTRACT

The project titled “Container Deployment Manager System” is designed to develop a scalable and efficient platform for managing and deploying containerized applications. The primary objective of this system is to simplify the process of container deployment, monitoring, and management by providing users with a centralized and automated interface. It eliminates the complexity of manual container handling and enables users to efficiently control application deployment environments.

This system allows users to create, start, stop, and manage containers seamlessly using containerization technology such as Docker. It provides real-time insights into container status, port configurations, and system performance, ensuring better resource utilization and operational transparency. Users can interact with the system through a user-friendly interface, making it accessible even for those with limited knowledge of container orchestration.

In addition to benefiting developers, the system also serves system administrators by offering better control over application deployment workflows. It reduces deployment time, minimizes human error, and enhances system reliability. The platform can be extended to support advanced features such as multi-container management, logging, and automated deployment pipelines.

The system is developed using modern full-stack technologies, integrating frontend, backend, and containerization tools to

ensure high performance, scalability, and reliability. The backend handles container lifecycle operations and communication with the container engine, while the frontend provides an interactive dashboard for managing deployments. The database maintains records of containers, configurations, and user activities securely.

In today's rapidly evolving technological landscape, containerization has become a key component of modern software development and DevOps practices. This project aims to provide a simplified yet powerful solution that bridges the gap between developers and container technologies, enabling efficient deployment, improved productivity, and streamlined application management.

INTRODUCTION

This project focuses on developing a comprehensive and user-friendly **Container Deployment Manager System** using modern full-stack technologies. The primary goal of the system is to simplify and automate the process of deploying and managing containerized applications, providing an efficient alternative to manual and complex deployment methods.

The platform allows users to create, deploy, and manage containers seamlessly using containerization tools such as Docker. Users can monitor container status, configure ports, and control container operations such as start, stop, and restart through a centralized interface. The system ensures a smooth user experience with an intuitive dashboard and real-time updates.

For developers and system administrators, the platform provides an efficient management environment to handle multiple containers, track performance, and manage application deployments. This helps streamline development workflows, reduce manual effort, and minimize errors during deployment processes.

By leveraging a combination of frontend, backend, and containerization technologies, the project ensures high performance, scalability, and reliability. The backend handles container lifecycle operations and communication with the container engine, while the frontend provides an interactive and

responsive interface. The system can be further extended to include advanced features such as logging, monitoring, and automated deployment pipelines.

In today's technology-driven world, containerization has become a crucial part of modern software development and DevOps practices. This project aims to provide a simplified, efficient, and scalable solution that enhances productivity, improves deployment efficiency, and supports the growing demand for automated infrastructure management.

SYSTEM ANALYSIS

The System Analysis phase involves a detailed study of traditional application deployment methods and the design of a modern container-based solution. It focuses on identifying the limitations of manual deployment processes and developing an efficient, scalable, and user-friendly system using containerization and full-stack technologies.

A. Existing System

In traditional deployment systems, applications are manually installed and configured on physical machines or virtual servers. This process is time-consuming, error-prone, and difficult to manage, especially when dealing with multiple environments or dependencies.

Developers often face issues such as:

- Environment inconsistency (“works on my machine” problem)
- Complex setup and configuration
- Difficulty in scaling applications
- Lack of proper monitoring and control

Additionally, there is no centralized system to manage application deployments, logs, or runtime status, making the process inefficient for both developers and administrators.

B. Proposed System

The proposed **Container Deployment Manager System** provides a centralized platform to deploy, manage, and monitor containerized applications efficiently.

The system uses containerization technology such as Docker to package applications along with their dependencies, ensuring consistency across different environments.

It includes:

- A **user interface (dashboard)** for managing containers
- A **backend system** to handle container lifecycle operations
- Integration with the container engine for executing commands

Users can create, start, stop, and monitor containers, configure ports, and track their status in real time. The system ensures smooth communication between frontend, backend, and container services, providing a seamless deployment experience.

C. Advantages

The proposed container-based deployment system offers several advantages over traditional methods:

- **Faster Deployment**

Applications can be deployed instantly using pre-configured containers, reducing setup time.

- **Environment Consistency**

Containers ensure that applications run the same across all environments, eliminating compatibility issues.

- **Centralized Management**

All containers can be managed from a single dashboard, improving control and visibility.

- **Scalability**

The system allows easy scaling of applications by running multiple containers as needed.

- **Reduced Errors**

Automation minimizes manual intervention, reducing configuration and deployment errors.

- **Efficient Resource Utilization**

Containers are lightweight and use system resources more efficiently compared to traditional setups.

- **Real-Time Monitoring**

Users can track container status, logs, and performance in real time.

OBJECTIVE AND SCOPE OF THE PROJECT

Objectives

The main objective of this project is to design and develop an efficient **Container Deployment Manager System** using modern full-stack technologies. The system aims to simplify the deployment and management of containerized applications while improving efficiency for developers and system administrators.

The key objectives include:

- To design and develop a container management system using full-stack technologies.
- To enable users to create, deploy, and manage containers using tools such as Docker.
- To provide real-time monitoring of container status, ports, and performance.
- To implement container lifecycle operations such as start, stop, and restart.
- To build a centralized dashboard for managing multiple containers efficiently.

Scope

The scope of the project defines the functionalities and boundaries of the **Container Deployment Manager System**. The system is designed to assist developers and administrators in managing containerized applications through a centralized and interactive platform.

- The system provides a simple and user-friendly interface for managing containers.
- Users can create and deploy containers with predefined configurations.
- The platform allows users to start, stop, restart, and remove containers easily.
- It provides real-time updates on container status and running processes.
- Users can configure port mappings and monitor container accessibility.
- The system maintains records of container activities and configurations.
- The platform is accessible via web browsers, ensuring cross-device compatibility.
- The system can be extended to support multiple containers, microservices architecture, and advanced orchestration features.
- It is scalable and can be integrated with advanced tools like orchestration systems in future enhancements.

FUNCTIONAL REQUIREMENTS

- **User Registration**

Users must register on the platform by providing basic details such as name, email, and password to access the system.

- **User Login & Logout**

Registered users can securely log in using valid credentials to access container management features , Users can securely log out from the system

- **Container Creation**

Users can create and deploy containers using containerization tools such as Docker.

- **Container Management**

The system allows users to start, stop, restart, and delete containers as per their requirements.

- **Configuration Setup**

Users can configure container settings like port mapping, container name, and image selection before deployment.

- **Container Status Monitoring**

The system provides real-time updates on container status (running or stopped) and basic information.

- **Error Handling**

The system displays proper error messages in case of failed deployment or invalid configurations.

- **Technical Requirement**

The system works on a client-server architecture and requires an active internet connection for functioning.

- **INTERFACE REQUIREMENTS**

The system includes several user interfaces to make container deployment and management simple and efficient. These interfaces help users interact easily with the platform.

1. Login Page

- This page allows registered users to sign in using their username and password to access the system securely.
-

2. Registration Page

- New users can create an account by entering basic details such as name, email, and password.
-

3. Dashboard (Main Interface)

- The system provides options such as **Create, Start, Stop, Restart, and Delete Containers**, allowing users to manage containers easily using tools like Docker.
-

4. Container Management Page

- This page allows users to configure container settings such as container name, image selection, and port mapping before deployment.
-

5. Status & Monitoring Page

- This page displays important details such as container status (running/stopped), port information, and basic logs in a clear and user-friendly format.

USE CASE DESCRIPTION AND USE CASE DIAGRAM

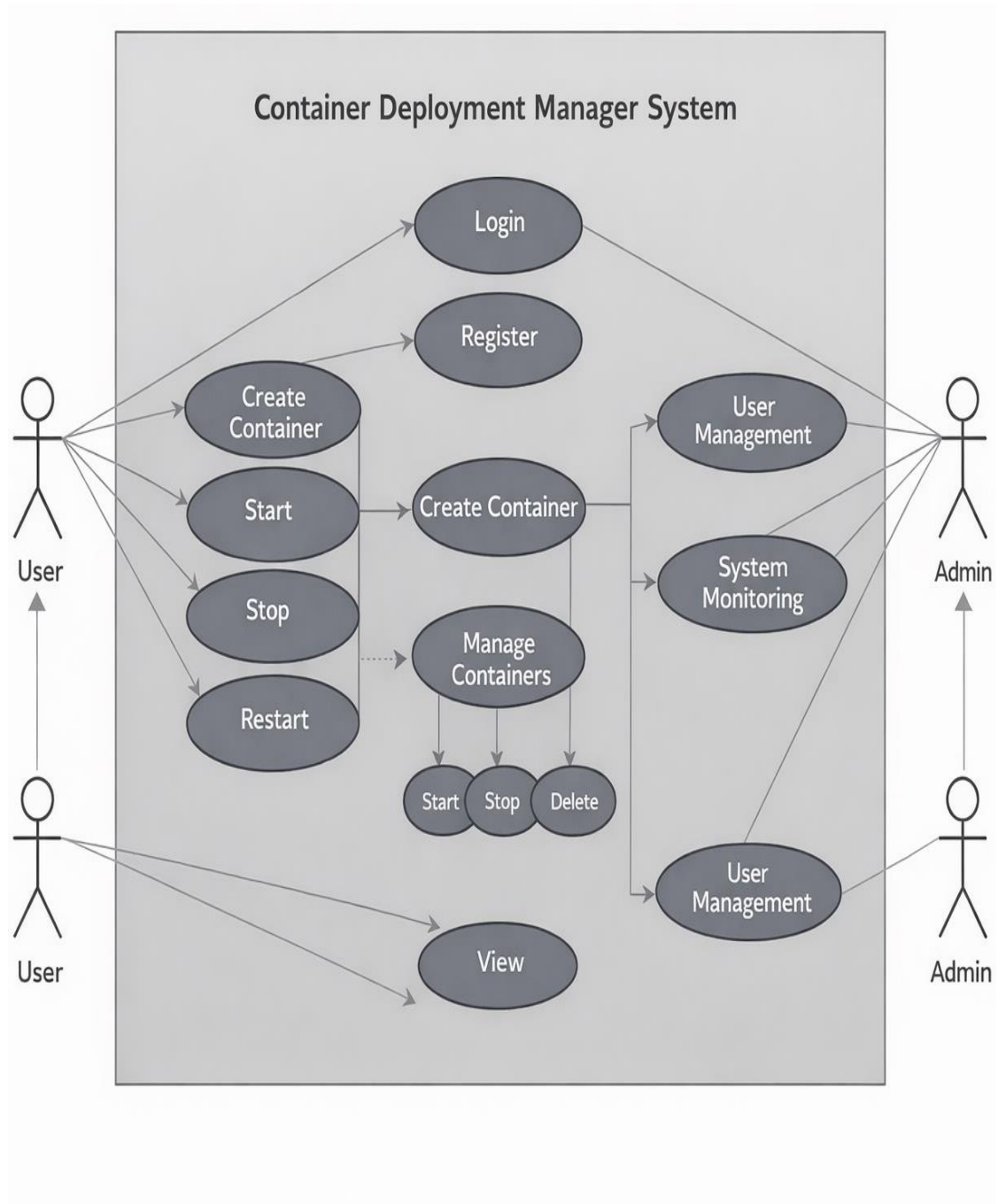
Container Deployment Manager System

Pre-condition:

User must be registered or logged into the system to access container management features.

Use Case Name	Description
User	A user who registers, logs in, and manages containers in the system
Login / Registration	User logs in or registers using email and password
Create Container	User creates and deploys a new container using Docker
Manage Containers	User can start, stop, restart, or delete containers
View Container List	User can view all containers along with their status and details
Configuration Setup	User configures container settings like port mapping and image selection
Monitoring	User checks container status, logs, and performance

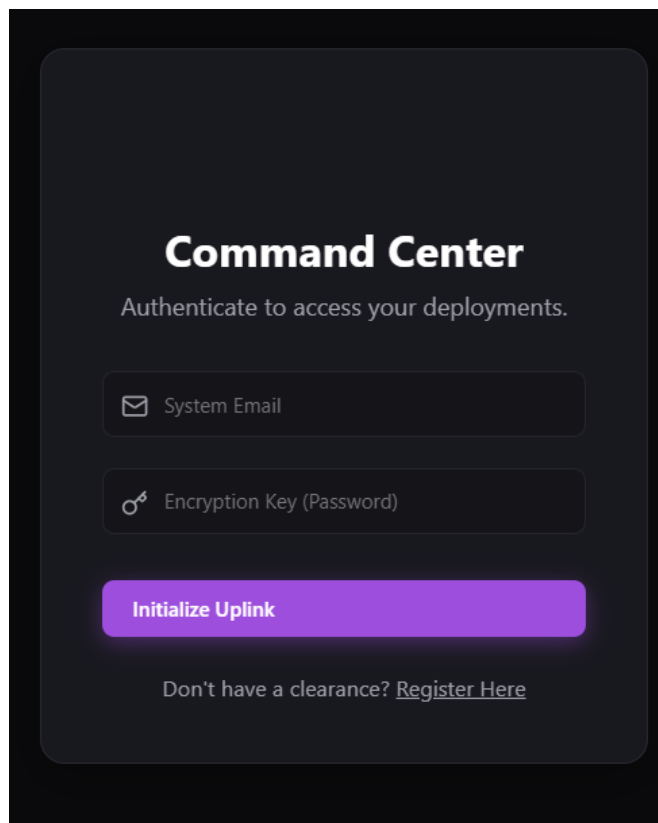
USER CASE DIAGRAM :



SYSTEM FLOW AND INTERFACE DESIGN

1. Registration
2. Login
3. Dashboard (Container List)
4. Container Configuration
5. Container Deployment & Management
6. Amin Monitoring & Output

1.Registration/Login

A dark-themed login interface for a 'Command Center'. It features a central dark gray rounded rectangle on a black background. Inside, the title 'Command Center' is in bold white text, followed by the subtitle 'Authenticate to access your deployments.' in a smaller white font. There are two input fields: the first is labeled 'System Email' with an envelope icon, and the second is labeled 'Encryption Key (Password)' with a key icon. Below these is a prominent blue button labeled 'Initialize Uplink'. At the bottom, a link reads 'Don't have a clearance? [Register Here](#)'.

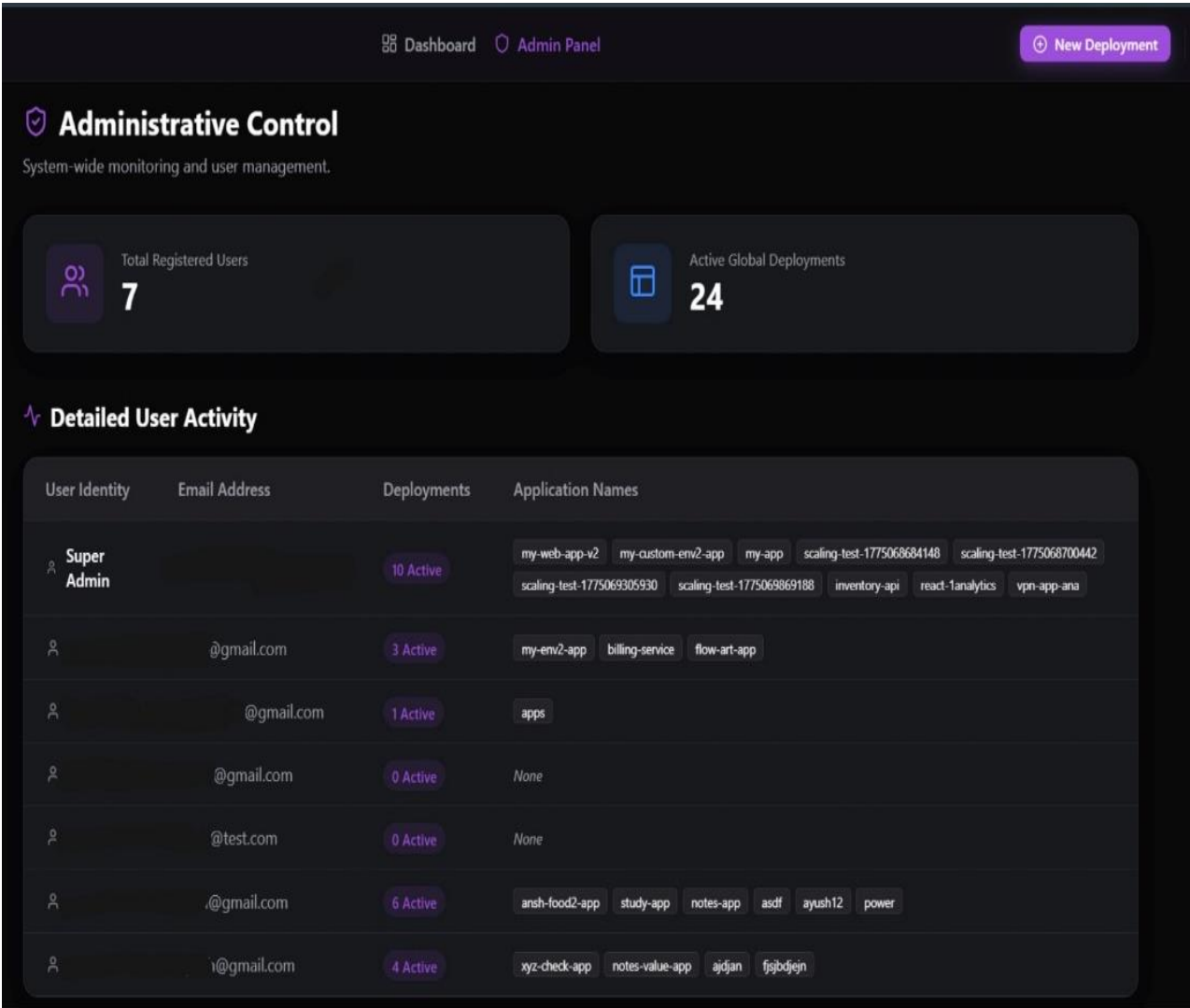
2. Dashboard (Container List)

The screenshot shows a dashboard titled "Deployments" with the subtitle "Manage your container environments." The dashboard features a top navigation bar with "Dashboard" and "Admin Panel" tabs, and a "New Deployment" button. The main content area displays a grid of six deployment cards. Each card shows the deployment name, status (e.g., "Blue Active", "Failed", "Stopped"), access port, and a "Deploy" button. The cards are: "vpn-app-ana" (Blue Active, Port: 8665), "react-1analytics" (Blue Active, Port: 8351), "inventory-api" (Blue Active, Port: 8732), "scaling-test-1775069869188" (Failed, Port: 8744), "scaling-test-1775069305930" (Failed, Port: 9385), and "scaling-test-1775068700442" (Stopped, Port: 8608). Each card also includes a "Post-Start Logic" section with a terminal output snippet.

3. Container Configuration and Container Deployment & Management

The screenshot shows a "Launch Deployment" form with the subtitle "Configure and initialize a new container environment." The form includes fields for "Application Name" (e.g., my-web-app), "Docker Image" (e.g., nginx), "Version / Tag" (latest), "Container Target Port" (80), "Git Repository URL" (https://github.com/user/repo), and "Post-Start Shell Script" (e.g., touch /app/ready.txt && echo 'Hello' > /app/hello.txt). There is also a section for "Environment Variables (Optional)" with a table for KEY and VALUE, and an "Add Var" button. A "Deploy Configuration" button is at the bottom right.

4. Amin Dashboard



HARDWARE & SOFTWARE TO BE USED

Hardware:

- Processor: Intel i3 or above
 - RAM: 4 GB or more
 - Storage: 500 GB HDD / 128 GB SSD
-

Software:

- Frontend: React.js
- Backend: Node.js, Express.js
- Database: MongoDB
- Containerization Tool: Docker
- Tools: Visual Studio Code, Git, Postman
- Browser: Google Chrome

Future Work of this Project

- Develop a mobile application for Android and iOS
- Add advanced monitoring (CPU, memory, real-time analytics)
- Support multiple containers and integrate with Kubernetes
- Implement CI/CD for automated deployment (GitHub integration)
- Add role-based access control (Admin/User) for better security

CONCLUSION

The project **“Container Deployment Manager System”** successfully demonstrates how modern technologies can be used to create an efficient, user-friendly, and scalable platform for managing containerized applications. The system simplifies the process of deploying, monitoring, and controlling containers, eliminating the complexity of traditional manual deployment methods.

By integrating frontend, backend, and containerization technologies such as Docker, the project ensures seamless communication between different components, providing a smooth and responsive user experience. The system allows users to efficiently manage container operations such as creation, start, stop, and monitoring through a centralized dashboard.

Overall, this project enhances productivity for developers and system administrators by reducing manual effort, minimizing errors, and improving deployment efficiency. It also provides a scalable foundation for future enhancements such as multi-container orchestration, advanced monitoring, and automated deployment pipelines.

Hence, the project achieves its primary goal of delivering a modern, reliable, and interactive container management solution that aligns with current DevOps practices and industry standards.

REFERENCES / BIBLIOGRAPHY

1. W3Schools – <https://www.w3schools.com>
(For learning HTML, CSS, JavaScript, and basic web technologies)
2. MDN Web Docs – <https://developer.mozilla.org>
(For detailed documentation on frontend and backend web technologies)
3. GeeksforGeeks – <https://www.geeksforgeeks.org>
(For backend development, system design, and DevOps concepts)
4. Stack Overflow – <https://stackoverflow.com>
(For debugging and resolving technical issues)
5. YouTube
(For practical tutorials on Docker, containerization, and full-stack development)
6. Official Documentation:
 - Node.js – <https://nodejs.org/en/docs>
 - React – <https://react.dev>
 - MongoDB – <https://www.mongodb.com/docs>
 - Express.js – <https://expressjs.com>
 - Docker – <https://docs.docker.com>